

[Algorithm]

퀵 정렬(Quicksort) 알고리즘

인하대학교 소프트웨어융합공학과

김종훈 교수



인하대학교
INHA UNIVERSITY

Contents

❖ 목차

- 1. 퀵 정렬
- 2. 빅오 표기법 복습
- 3. 퀵 정렬 구현



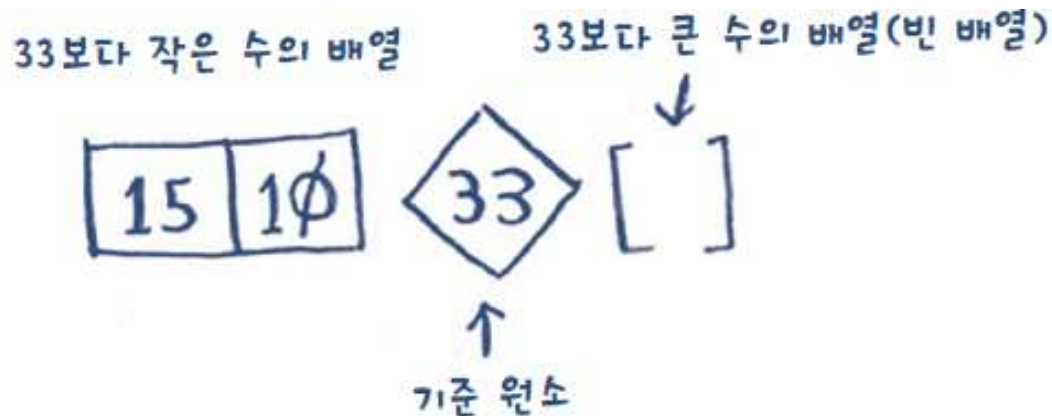
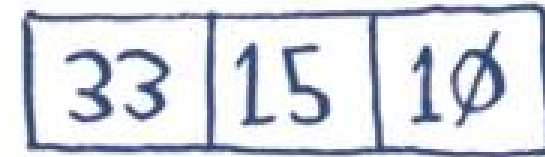
1. 퀵 정렬

❖ 퀵 정렬 (Quicksort)

■ 정렬 알고리즘

■ Ex) 원소가 세 개인 배열

- **기준 원소** (pivot) 선택하여 다른 원소를 이보다 크거나 작은 원소로 분류
 - 분할 (Partitioning)
- 하위 배열 (Sub-array) 에 대해 재귀적으로 퀵 정렬 호출하여 정렬
 - 결과 합쳐 전체 배열 정렬

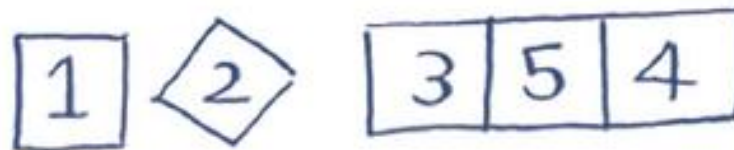
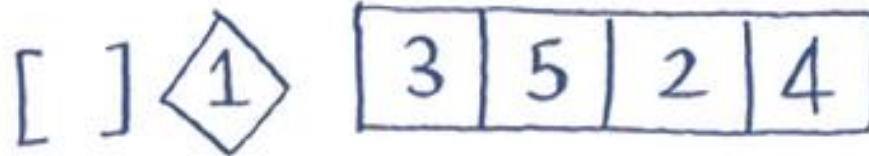


```
quicksort([15, 10]) + [33] + quicksort([])  
> [10, 15, 33] ← 정렬된 배열
```



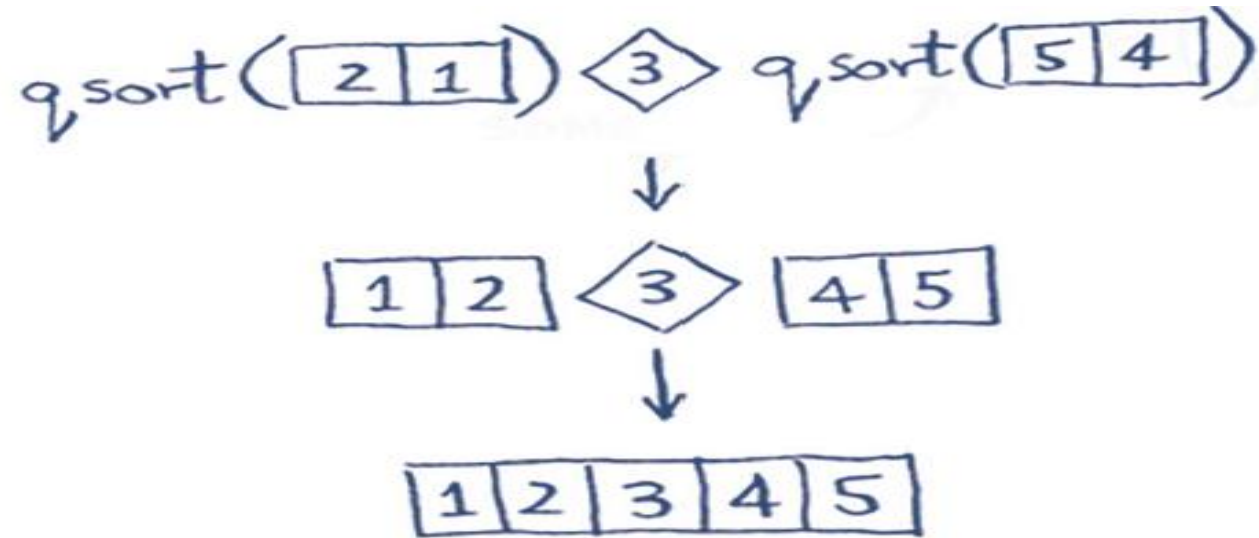
1. 퀵 정렬

- Ex) 원소가 다섯 개인 배열
 - 기준 원소에 따라 아래와 같이 분할

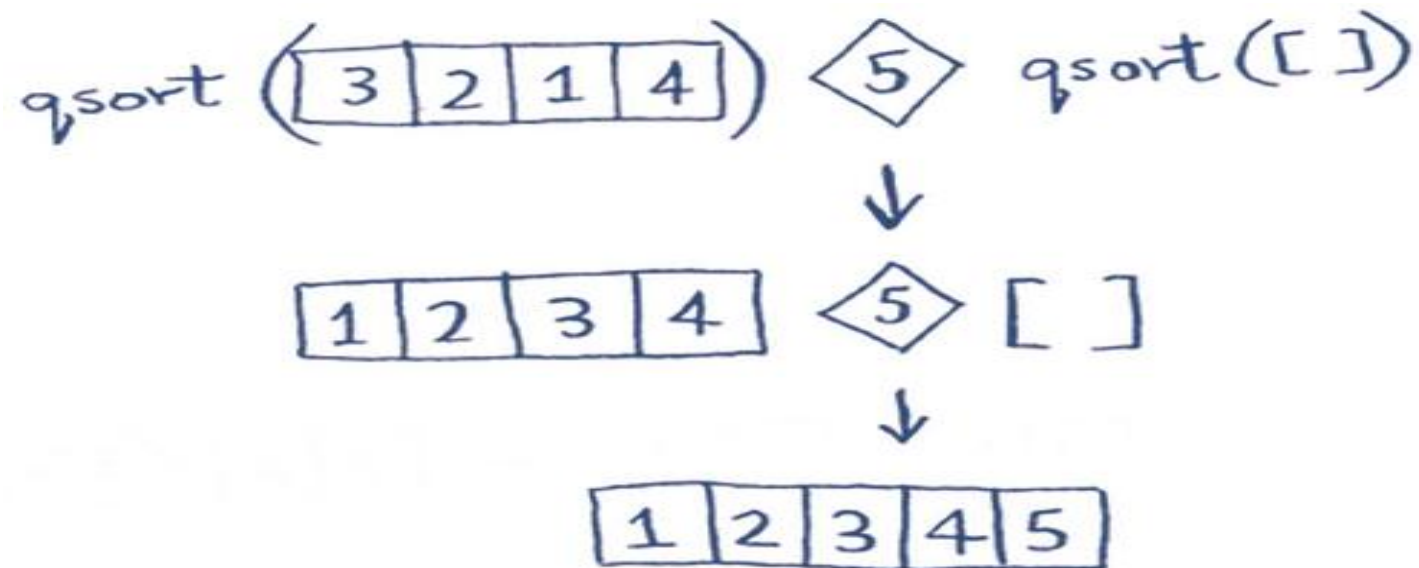


1. 퀵 정렬

- 하위 배열에 대해 퀵 정렬 호출 (기준 원소 3 경우)



- 각 하위 배열 정렬되면 합쳐서 전체 배열 정렬



1. 퀵 정렬

■ 코드로 구현

```
def quicksort(array):
```

```
    if len(array) < 2:
```

```
        return array
```

기본 단계: 원소의 개수가 0이나 1이면
이미 정렬되어 있는 상태

```
    else:
```

```
        pivot = array[0]
```

재귀 단계

```
        less = [i for i in array[1:] if i <= pivot]
```

기준 원소보다 작거나 같은 모든 원소로 이루어진 하위 배열

```
        greater = [i for i in array[1:] if i > pivot]
```

기준 원소보다 큰 모든 원소로 이루어진 하위 배열

```
        return quicksort(less) + [pivot] + quicksort(greater)
```

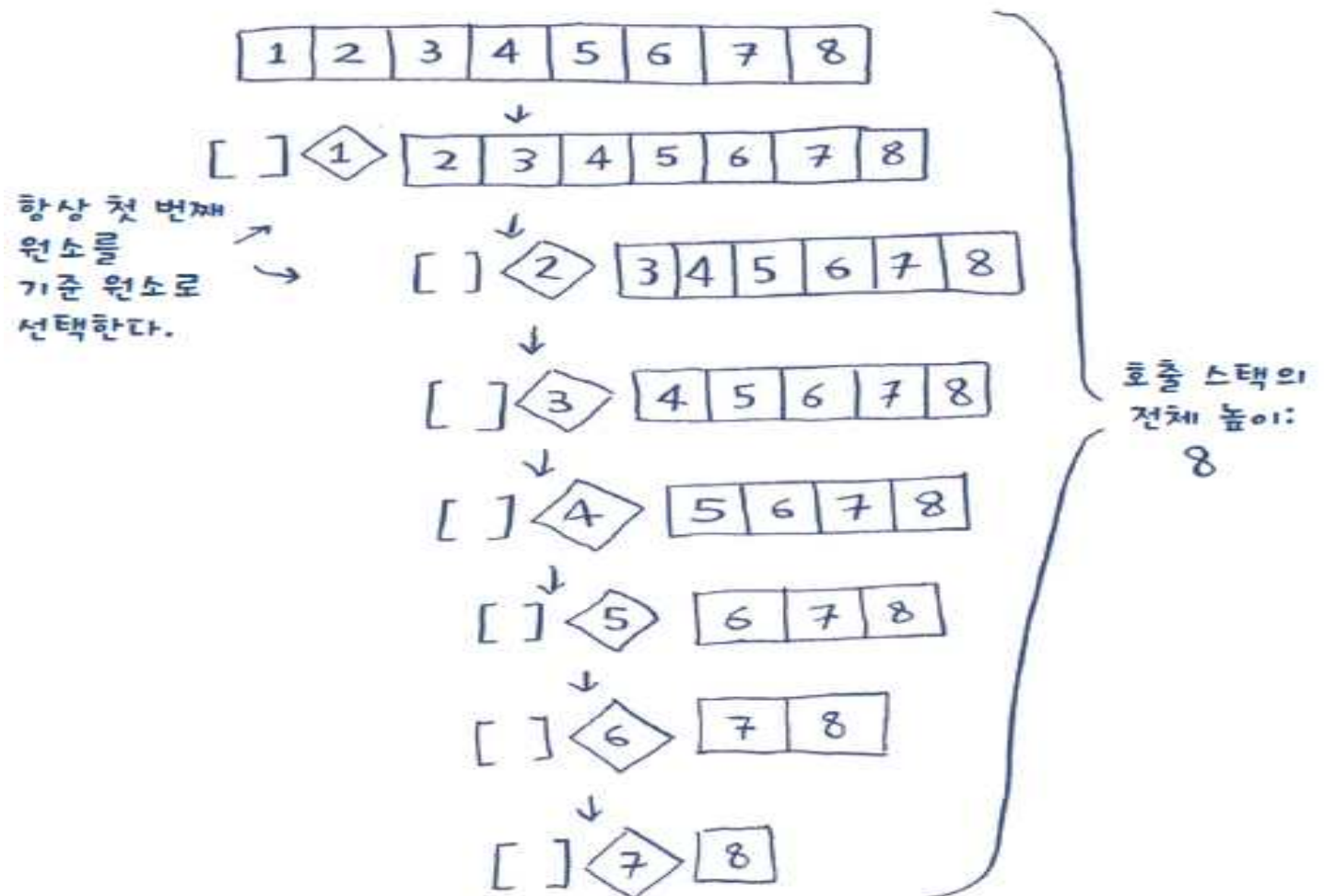
```
print quicksort([10, 5, 2, 3])
```



2. 빅오 표기법 복습

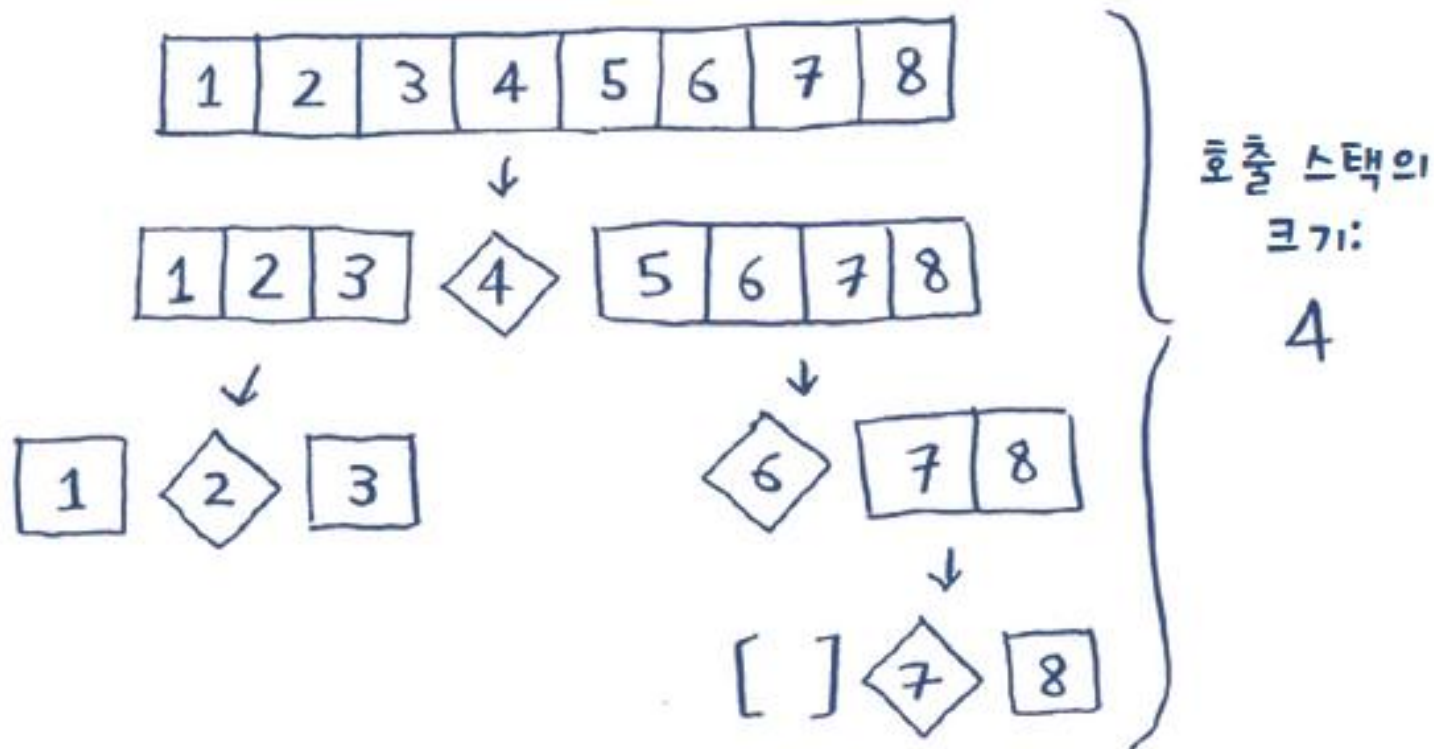
❖ 평균적인 경우와 최악의 경우 비교

- 퀵 정렬의 성능은 선택된 기준 원소에 크게 연관
 - Ex) 첫 번째 원소를 항상 기준 원소로 선택할 경우
 - $O(n)$



2. 빅오 표기법 복습

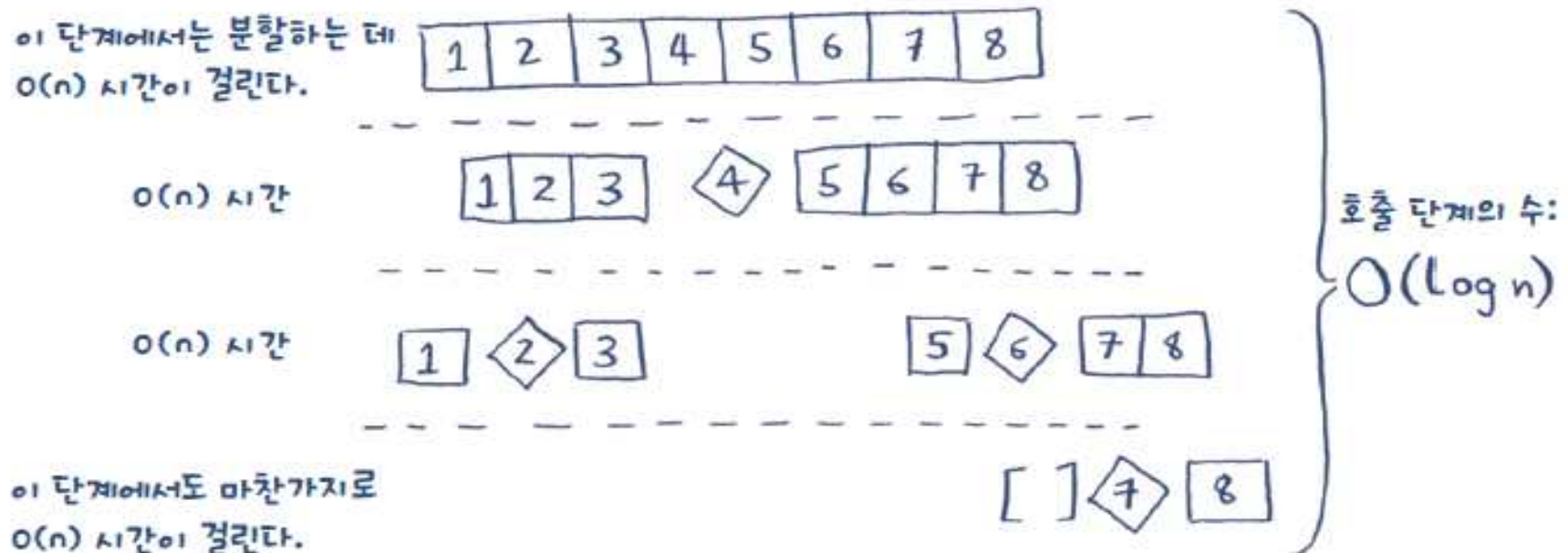
- Ex) 정가운데 원소를 항상 기준 원소로 선택할 경우
 - 기본 단계에 더 빨리 도달
 - $O(\log n)$



2. 빅오 표기법 복습

❖ 전체 알고리즘 시간

- 최선의 경우: $O(n) \times O(\log n) = O(n \log n)$ 시간
- 최악의 경우: $O(n) \times O(n) = O(n^2)$ 시간
- 스택 첫 번째 단계 및 이후의 각각의 모든 호출 스택
 - 기준 원소 선택 시 나머지 원소들은 하위 배열로 나누어짐
 - 8개 원소 모두 기준 원소와 비교 필요
 - $O(n)$ 시간 소요



2. 빅오 표기법 복습

❖ Ex) 초당 10회 연산을 하는 경우

예제 알고리즘	이진 탐색	단순 탐색	퀵 정렬	선택 정렬	외판원 문제
배열 크기	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(n!)$
10	0.3초	1초	3.3초	10초	4.2일
100	0.6초	10초	66.4초	16.6분	2.9×10^{149} 년
1000	1초	100초	996초	27.7시간	1.27×10^{2559} 년

- 병합 정렬 (Merge Sort)
 - $O(n \log n)$ 시간
- 퀵 정렬은 최악의 경우 $O(n^2)$ 까지 가능



2. 빅오 표기법 복습

❖ 합병 정렬과 퀵 정렬 비교

■ Ex) 리스트에 있는 모든 원소를 출력하는 함수

- 원소 출력 전 1초 대기하도록 변경
- 두 함수를 사용하여 5개의 원소 가지는 리스트 출력

```
def print_items(list):  
    for item in list:  
        print item
```

```
from time import sleep  
def print_items2(list):  
    for item in list:  
        sleep(1)  
        print item
```

- 두 함수 모두 빅오 표기법으로는 $O(n)$ 실행 시간
- 실제로는 전자가 훨씬 빠름



print_items: 2 4 6 8 10

print_items2: <[H71]> 2<[H71]> 4<[H71]> 6<[H71]> 8<[H71]> 10

2. 빅오 표기법 복습

- 빅오 표기법을 사용한다는 것은:

$C * n$
어떤 일정 시간 ↗

- 상수 (Constant)

- 알고리즘이 소비하는 특정 시간
- print_items 함수: 10 milliseconds x n 시간
- print_items2 함수: 1 second x n 시간
- 두 알고리즘이 서로 다른 빅오 표기법 시간 가질 경우 주로 무시
- 퀵 정렬과 합병 정렬 비교의 경우 상수로 인한 차이 발생
 - » 퀵 정렬이 더 작은 상수 가짐
 - » $O(n \log n)$ 으로 실행 시간 동일하다면 퀵 정렬이 더 빠름



3. 퀵 정렬 구현

■ 코드로 구현

```
# 퀵 정렬
# 입력: 리스트 a
# 출력: 없음(입력으로 주어진 a가 정렬됨)
# 리스트 a의 어디부터(start) 어디까지(end)가 정렬 대상인지
# 범위를 지정하여 정렬하는 재귀 호출 함수
def quick_sort_sub(a, start, end):
    # 종료 조건: 정렬 대상이 1개 이하면 정렬할 필요 없음
    if end - start <= 0:
        return
    # 기준 값을 정하고 기준 값에 맞춰 리스트 안에서 각 자료의 위치를 맞춤
    # [기준 값보다 작은 값들, 기준 값, 기준 값보다 큰 값들]
    pivot = a[end] # 편의상 리스트의 마지막 값을 기준 값으로 정함
    i = start
    for j in range(start, end):
        if a[j] <= pivot:
            a[i], a[j] = a[j], a[i]
            i += 1
    a[i], a[end] = a[end], a[i]
    # 재귀 호출 부분
    quick_sort_sub(a, start, i - 1) # 기준 값보다 작은 그룹을 재귀 호출로 다시 정렬
    quick_sort_sub(a, i + 1, end) # 기준 값보다 큰 그룹을 재귀 호출로 다시 정렬

# 리스트 전체(0 ~ len(a)-1)를 대상으로 재귀 호출 함수 호출
def quick_sort(a):
    quick_sort_sub(a, 0, len(a) - 1)

d = [6, 8, 3, 9, 10, 1, 2, 4, 7, 5]
quick_sort(d)
print(d)
```





Thank You